

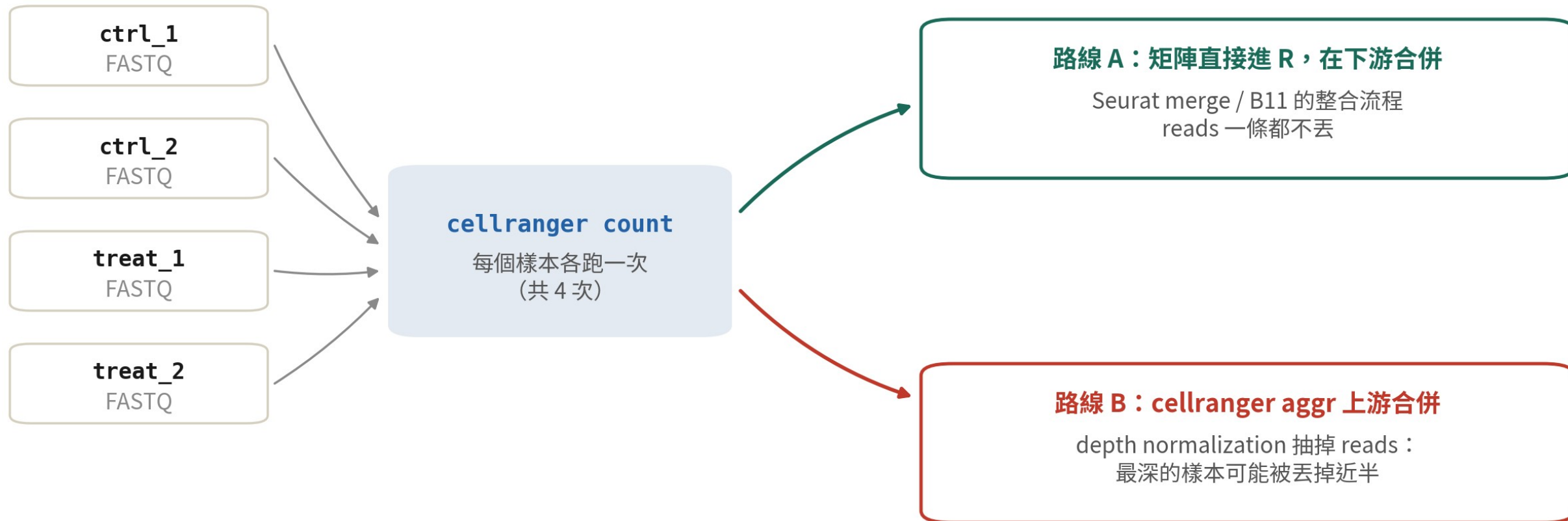
B4 B 系列

多樣本與多模態： aggr、multi、Feature Barcode

多樣本與 CITE-seq / hashing 的輸出，你都看得懂、選得對指令。

約 35 分鐘 | 前置：B3 | 資料：10x 官方 multi / hashing 範例（只判讀，不要求自己跑）

四個樣本，兩條路



四個樣本要合併分析——你選哪條路？代價各是什麼？

兩條路都能把四個樣本變成一份分析——差別在 reads 的命運

本集的起點

合併的方式選錯， 代價要到幾週後才浮現。

aggr 預設會把 reads 抽掉，而且抽掉的不會回來。很多人是在需要原始深度的那一天才發現。

這一集你會學到

- 1 分清 count、aggr、multi 各在什麼時候用，說得出理由
- 2 說出 Feature Barcode 資料（ADT / HTO）在矩陣裡長什麼樣
- 3 看懂 hashing demultiplex 的三種輸出：singlet、doublet、negative

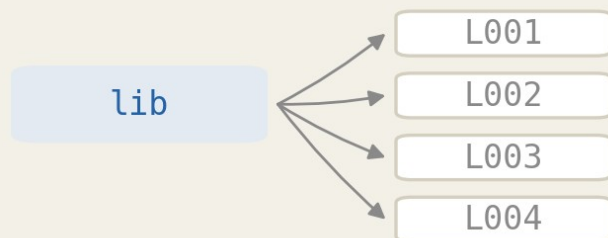
01

三種「多」，先分清楚

多 lane、多樣本、多模態，處理方式完全不同

多 lane、多樣本、多模態

多 lane



同一個 library 分幾條 lane 定序
count 自動合併 (B2 講過)

不用做任何事

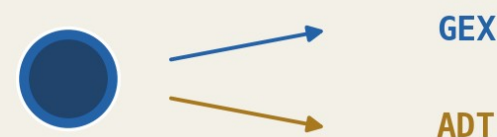
多樣本



不同人、不同條件的樣本
各自跑 count，再決定怎麼合

本集前半：aggr 或 R 端

多模態



同一批細胞、兩種 library：
RNA+表面蛋白 (Feature Barcode)

本集後半：multi / FB

多 lane 是假問題，count 自己會合；剩下兩種才是本集的主題

多樣本合併：上游還是下游？

這是策略選擇，不是語法選擇

下游合併

各跑 **count**，矩陣進 **R** 再合

- reads 一條不丟，資訊全保留
- 批次效應交給 B11 的整合方法
- 現代分析的預設路線

上游合併

cellranger aggr 合成一包

- 輸出單一矩陣 + 單一 web_summary
- 預設做 depth normalization——會丟 reads
- 適合快速總覽、需要單一矩陣的工具

本集核心觀念

合併矩陣很容易， 合併「可比性」才是難題。

aggr 用丟 reads 換可比性；下游整合用統計模型換。B11 會講為什麼後者是主流。

02

cellranger aggr 實際操作

一張 CSV、一個指令、一個要想清楚的參數

aggregation CSV : 兩欄就夠

```
cat> aggr.csv << 'EOF'  
sam ple_id,m olecule_h5  
ctrl_1,rns/ctrl_1/outs/m olecule_info.h5  
ctrl_2,rns/ctrl_2/outs/m olecule_info.h5  
treat_1,rns/treat_1/outs/m olecule_info.h5  
treat_2,rns/treat_2/outs/m olecule_info.h5  
EOF
```

aggr 吃的是每個 count 輸出裡的 molecule_info.h5 ，不是矩陣資料夾

跑 aggr：關鍵在 --normalize

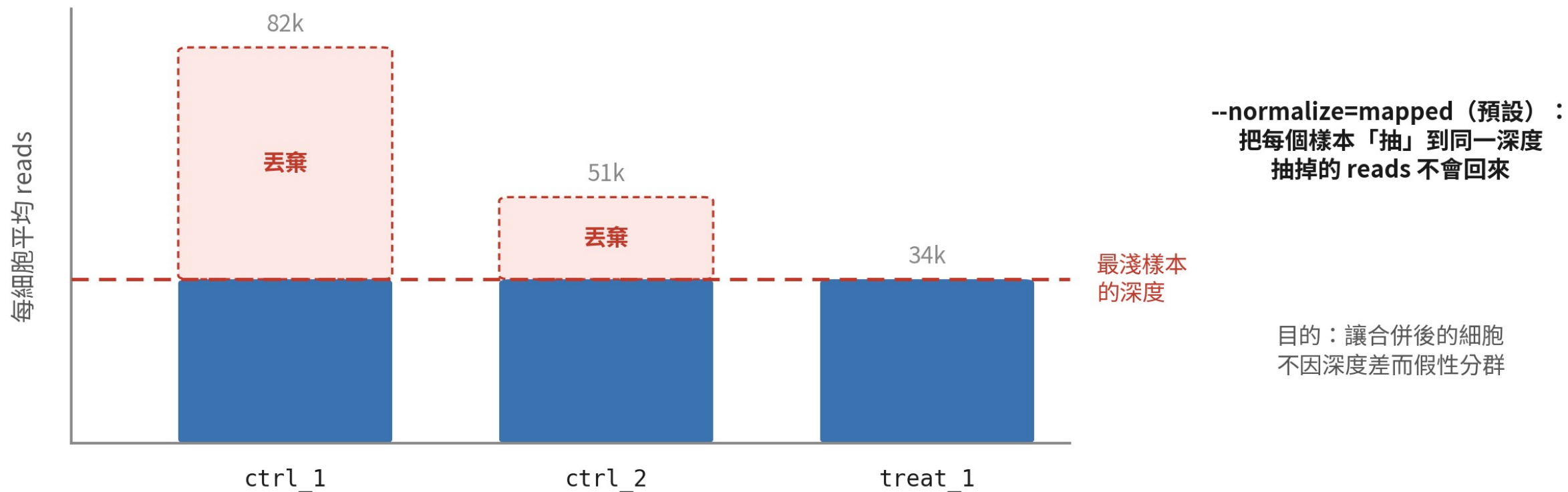
```
cellranger aggr \  
  --id= all_samples \  
  --csv= aggr.csv \  
  --normalize=mapped # 預設。 none = 保留全部 reads
```

Outputs:

- Run summary HTML: /.../all_samples/outs/web_summary.html
- Aggregation CSV: /.../all_samples/outs/aggregation.csv
- Filtered feature-barcode matrices:
 /.../outs/count/filtered_feature_bc_matrix

mapped = 把每個樣本抽到同一深度； none = 不抽樣，全部保留

depth normalization 在做什麼



模擬資料

向下看齊：所有樣本被抽到最淺那個樣本的深度，多的 reads 直接丟掉

aggr 的輸出： barcode 多了字尾

aggregation CSV

```
sample_id,molecule_h5  
ctrl_1,...molecule_info.h5  
ctrl_2,...molecule_info.h5  
treat_1,...molecule_info.h5  
treat_2,...molecule_info.h5
```

吃的是 molecule_info.h5，
不是矩陣資料夾

cellranger
aggr

一份合併矩陣

AAACCCAAGAAACACT	-1
TGCACGGTCAGGACGT	-1
AAACCCAAGAAACACT	-2
CTTCAATTCGGAGATG	-3

字尾 -1 / -2 / -3 / -4：
CSV 裡的第幾列（樣本）

同一段 barcode 序列可能
出現在不同樣本——
靠字尾才分得開

-1、-2、-3 ... 對應 CSV 的列序；下游靠這個字尾知道細胞來自哪個樣本

aggr：這一節的判斷

何時仍有用



要一份總覽
web_summary、
或工具只吃單一矩陣
時

用的時候



想清楚 --
normalize；多數下
游分析可以設 none

主流做法



各跑 count，R 端
合併+ B11 整合

03

cellranger multi 是什麼

一個 run、多種 library、多個樣本的總指揮

count 還是 multi ?

只有 GEX

一個樣本一條 library

cellranger count

B3 的世界

GEX+抗體 (ADT / HTO)

加一份 feature reference

count --libraries / multi

兩條路都通

CellPlex 多重化 (CMO)

要按樣本拆 barcode

cellranger multi

只能 multi

5' VDJ (TCR / BCR)

GEX+VDJ 要配對

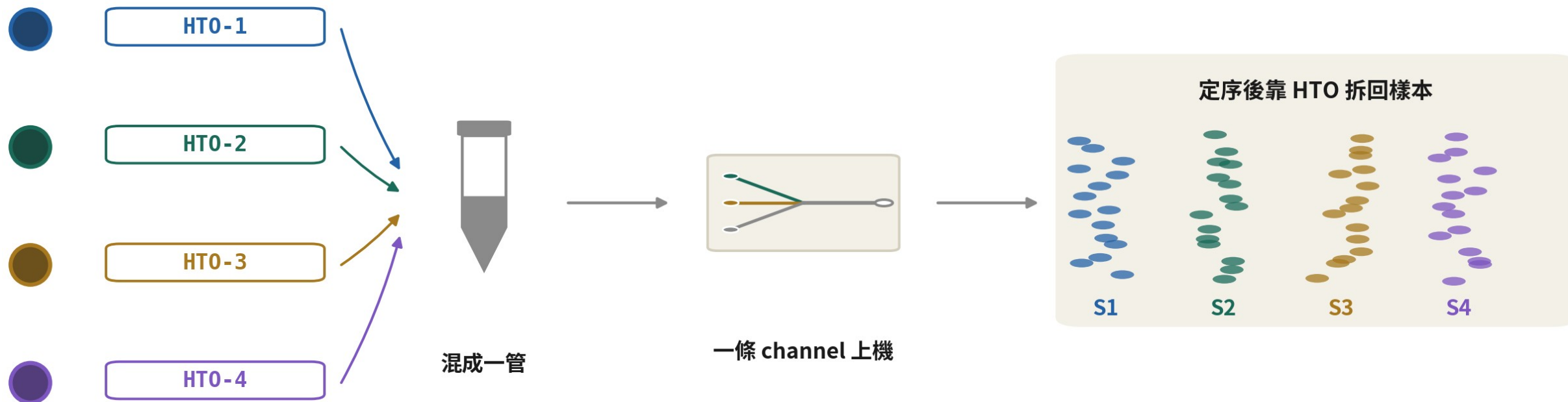
cellranger multi

只能 multi

判斷準則：資料要「按樣本拆開」或含 VDJ → 走 multi；其餘 count 就夠

要按樣本拆 barcode、或含 VDJ 的實驗，只能走 multi

為什麼要多重化（multiplexing）



每個樣本先用不同的
hashtag 抗體標記

好處：省試劑、同一批次零批次效應、跨樣本 doublet 可以辨識

先標記、再混樣、一條 channel 上機——定序完再用標籤拆回來

multi config CSV (上) : 資源與 libraries

```
# multi_config.csv — 一個檔案分好幾個 [區塊]  
[gene-expression]  
reference,refs/refdata-gex-GRCh38-2024-A  
create-bam,false  
  
[libraries]  
fastq_id,fastqs,feature_types  
pbmc_gex,fastq/gex,Gene Expression  
pbmc_cmo,fastq/cmo,Multiplexing Capture
```

libraries 區塊宣告每組 FASTQ 是哪一種 library——這裡是 GEX 加 CellPlex 的 CMO

multi config CSV (下) : samples 區塊

```
[samples]
sample_id,cm o_ids
donor_A,CM 0301
donor_B,CM 0302
donor_C,CM 0303
donor_D,CM 0304
```

```
$ cellranger multi --id=pbm_c_multi --csv=multi_config.csv
...
Outputs:
- per-sample outputs: /.../pbm_c_multi/outs/per_sample_outs
```

samples 區塊告訴 multi 「哪個標籤是哪個樣本」——漏了它就拆不了樣本

multi 的輸出: per_sample_outs

```
pbmc_multi/outs/
```

```
├─ multi/count/raw_feature_bc_matrix/
```

← 全部細胞，還沒拆樣本

```
├─ per_sample_outs/
```

```
│   └─ donor_A/
```

```
│       └─ web_summary.html
```

← 每個樣本自己一份報告

```
│       └─ count/sample_filtered_feature_bc_matrix/
```

← 下游要讀的就是它

```
│   └─ donor_B/ ...
```

```
│   └─ donor_C/ ...
```

```
└─ config.csv
```

← 你的設定檔會被存一份

count 的輸出一個樣本一個資料夾；multi 把「同一 run 拆出的樣本」放在 **per_sample_outs**

每個樣本一個資料夾、一份 `web_summary`；下游讀 `sample_filtered_feature_bc_matrix`

multi：這一節做了什麼

一張 config



[gene-expression]、
[libraries]、
[samples] 三個必
懂區塊

一次拆完



CellPlex 的樣本歸
屬在上游就拆好

按樣本輸出



per_sample_outs
一人一份，直接接
B5 的讀入

04

Feature Barcode：把蛋白變成序列

CITE-seq 的 ADT、cell hashing 的 HTO，原理是同一個

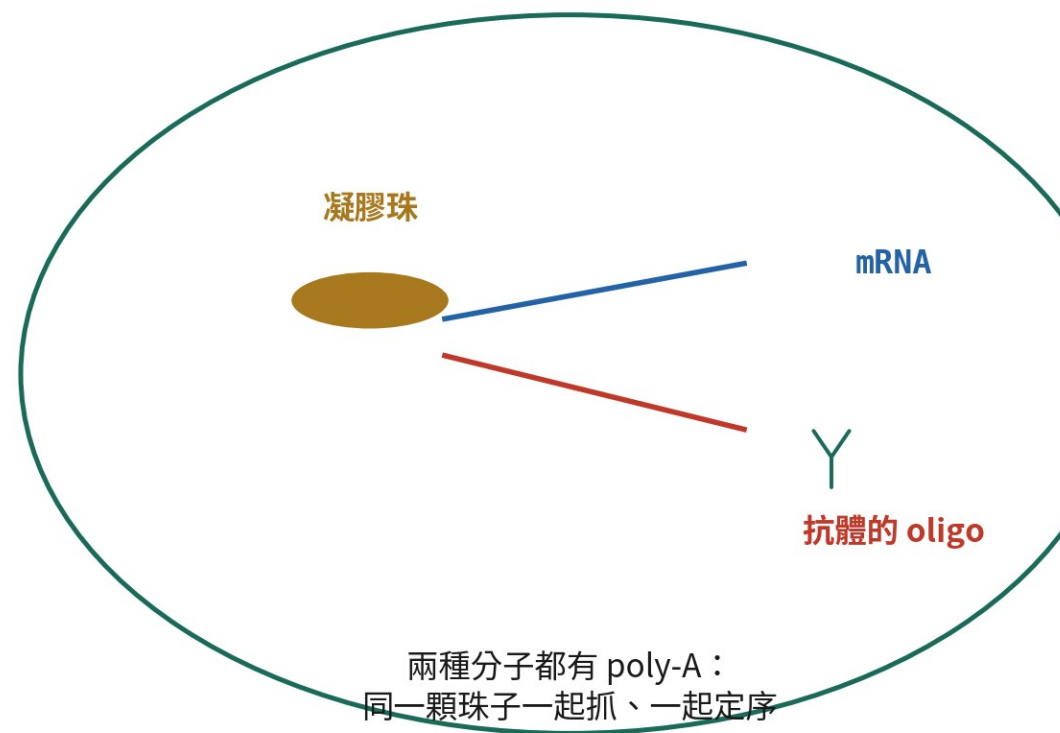
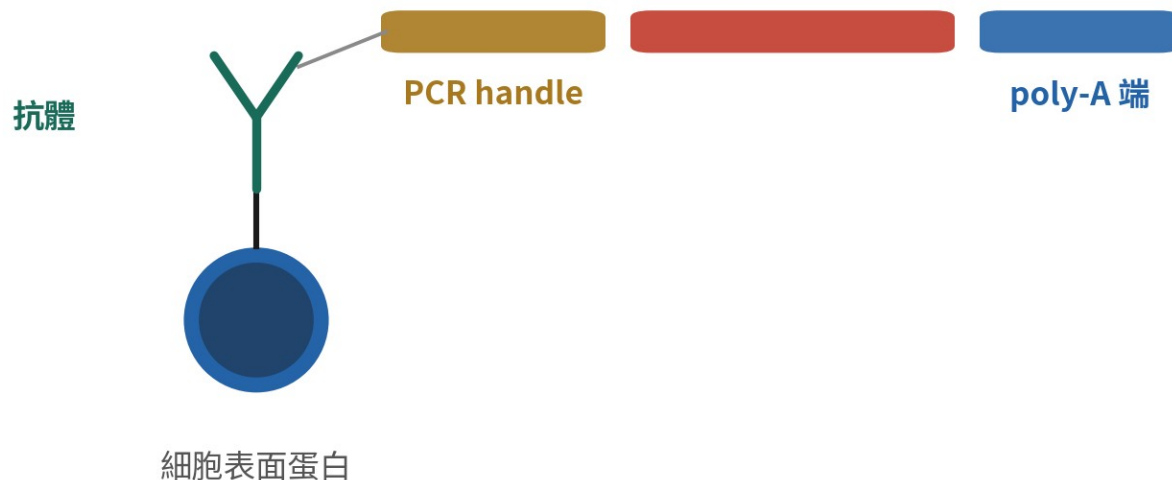
一條 oligo，把抗體變得可定序

ADT = 表面蛋白訊號 (CITE-seq, Stoeckius 2017)

HTO = 樣本標籤 (cell hashing, Stoeckius 2018)

抗體上綁一條人工 DNA：
中間那段序列就是這個抗體的身分證

Feature Barcode (15 nt)



抗體綁著人工 DNA：定序儀讀到那段 barcode，就等於「看到」了那個蛋白

count 也能跑 FB : libraries CSV

```
cat> libraries.csv << 'EOF'  
fastqs,sample,library_type  
fastq/gex,pbm_c_gex,Gene Expression  
fastq/adt,pbm_c_adt,Antibody Capture  
EOF
```

```
cellranger count --id=pbm_c_citeseq \  
  --transcriptome=refs/refdata-gex-GRCh38-2024-A \  
  --libraries=libraries.csv \  
  --feature-ref=feature_ref.csv \  
  --create-bam=false
```

不用 --fastqs 了：兩種 library 都寫進 libraries.csv，各自標明類型

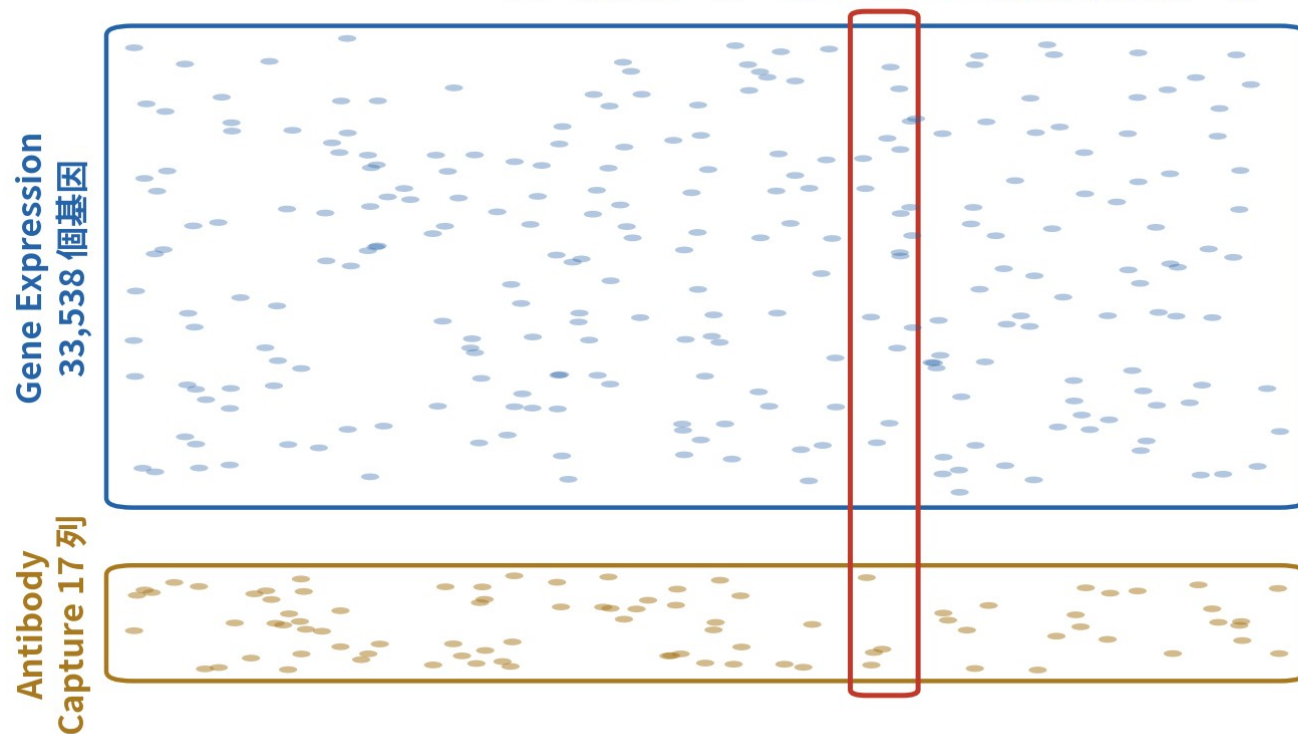
feature reference CSV 解剖

```
cat> feature_ref.csv << 'EOF'
id,name,read,pattern,sequence,feature_type
CD3,CD3_TotalB,R2,5PNNNNNNNNNNN(BC),AACAAAGACCCTTGAG,Antibody Capture
CD19,CD19_TotalB,R2,5PNNNNNNNNNNN(BC),CTGGGCAATTACTCG,Antibody
Capture
HT01,Hashtag1,R2,5PNNNNNNNNNNN(BC),GTCAACTCTTTAGCG,Antibody Capture
HT02,Hashtag2,R2,5PNNNNNNNNNNN(BC),TGATGGCCTATTGGG,Antibody Capture
EOF
```

sequence 欄是每個抗體的身分證——來源只能是試劑說明書，逐字核對

輸出矩陣：一個 barcode、兩種 feature

同一顆細胞=同一個 barcode，兩個矩陣各佔一欄



細胞 barcode（欄，兩個矩陣完全相同）

Gene Expression 與 Antibody Capture 疊在同一份矩陣裡，靠 features.tsv 第三欄區分

features.tsv 的第三欄

每一列標著 Gene Expression
或 Antibody Capture

Read10X 讀進來

自動拆成一個 list：
兩個矩陣、兩個名字

進 Seurat

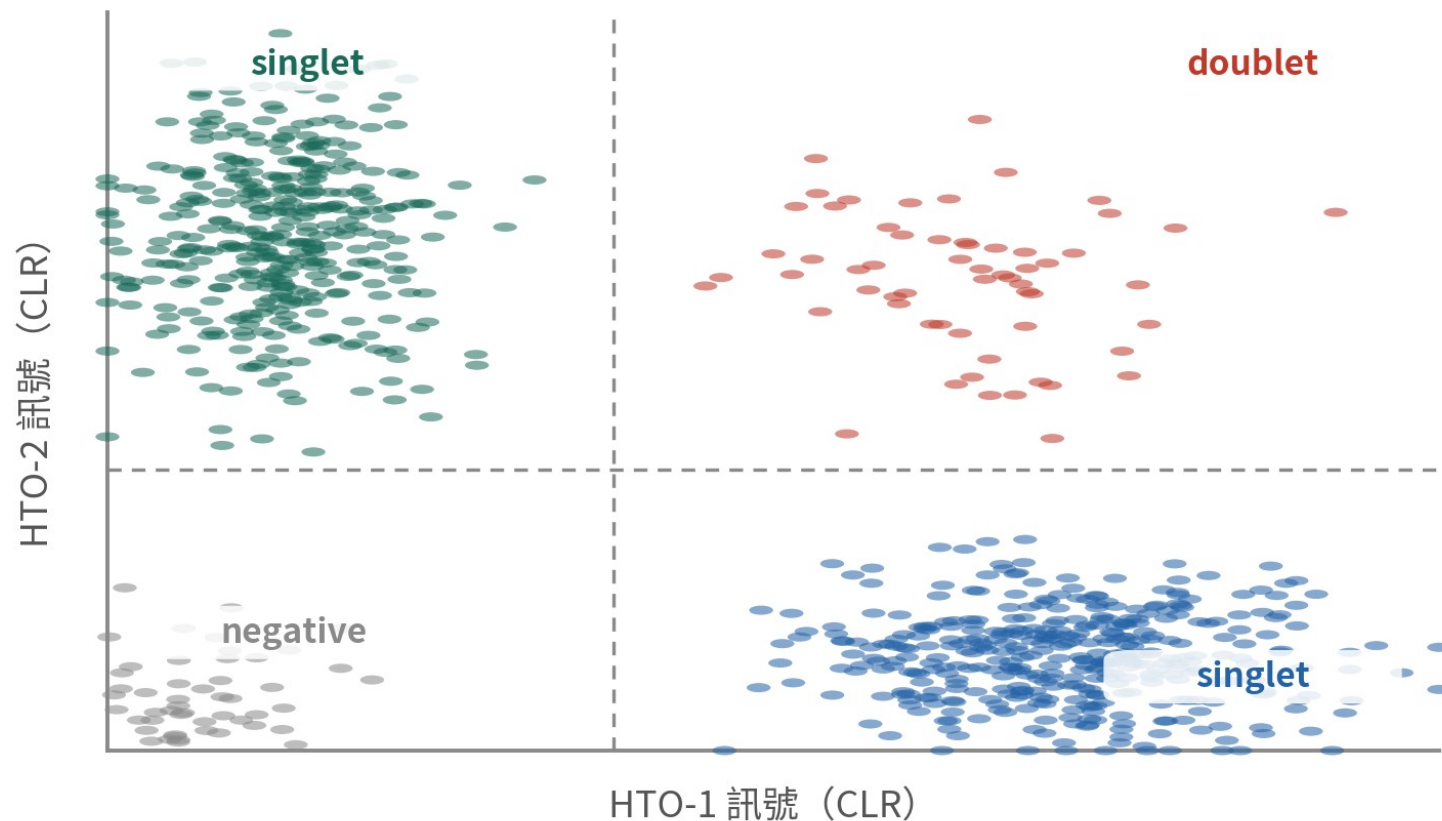
RNA assay + ADT assay，
同一個物件、同一批細胞

05

hashing 的 demultiplex 與下游承接

HTODemux 在 R 端做；矩陣怎麼讀進 Seurat

demultiplex 的三種判定



singlet

一種 HTO 高、其他低
→ 樣本歸屬明確，留下

doublet

兩種以上 HTO 都高
→ 跨樣本雙細胞，丟掉

negative

每種 HTO 都低
→ 沒染上標籤，另外討論

模擬資料

一種 HTO 獨高 = singlet ; 多種都高 = doublet ; 全都低 = negative

Seurat 讀入：一個物件、兩個 assay

```
library(Seurat)
set.seed(1234)

d <- Read10X("outs/filtered_feature_bc_matrix/")
names(d) # 有 FB 時回傳的是 list
pbmc <- CreateSeuratObject(counts = d[["Gene Expression"]])
pbmc[["HTO"]] <- CreateAssayObject(
  counts = d[["Antibody Capture"]])
pbmc
```

An object of class Seurat
33555 features across 7865 samples within 2 assays
Active assay: RNA (33538 features, 0 variable features)
1 other assay present: HTO

RNA 與 HTO 住在同一個物件裡，共用同一批細胞 barcode

HTODemux : 把細胞判回樣本

```
pbm c <- Norm alizeData (pbm c, assay = "HT0 ",  
                        norm alization.m ethod = "CLR")  
pbm c <- HT0Dem ux (pbm c, assay = "HT0 ",  
                   positive.quantile = 0.99)  
table (pbm c$HT0_classification.g lobal)
```

```
DoubletNegative Singlet  
680    346    6839
```

結果寫進 metadata ; 常規做法是 subset 出 Singlet 再往下游走

到這裡，你手上有什麼

拆好的樣本



每顆細胞都知道自己
來自誰

兩層資訊



RNA 與蛋白（或標
籤）在同一個物件

接 B5



filtered matrix 在
手，下游從 QC 開
始

06

踩雷區

三個大雷、兩個小雷，都給你看後果

雷一： aggr 之後才想要原始深度

雷是什麼

用預設 `--normalize=mapped` 跑完 aggr，之後的分析都從合併輸出出發。



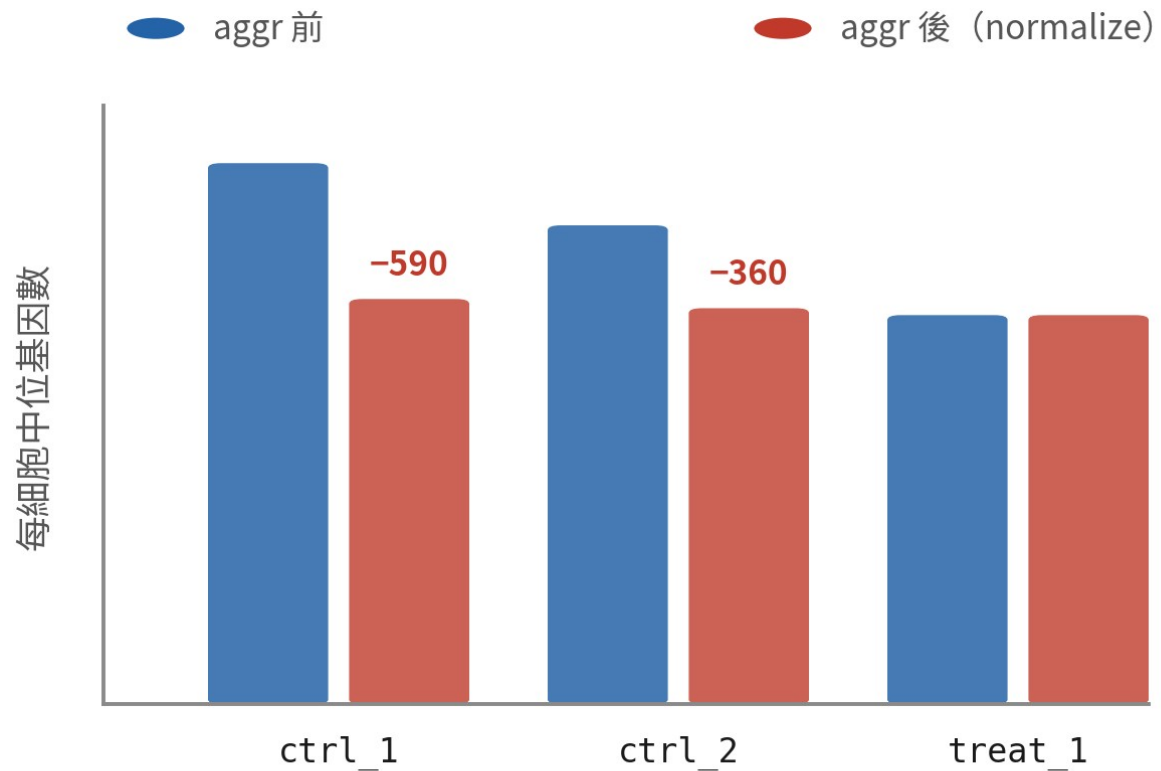
為什麼會踩

預設值看起來無害，「正規化」聽起來還像好事。

踩了會怎樣

最深的樣本被抽掉近半 reads；想做飽和度或稀有轉錄本分析時，只能回頭重跑。

被抽掉的 reads 不會回來



被抽掉的 reads 不在任何輸出裡

想做飽和度、稀有轉錄本、深度相關的分析？

aggr 的輸出救不回來——

得回到每個樣本的 count 輸出重來

一行檢查：web_summary 的 aggregation 表
看每個樣本被抽掉的比例

模擬資料

一行檢查：aggr 的 web_summary 有每個樣本被抽掉比例的表

雷二： feature reference 序列打錯

雷是什麼

sequence 欄手動輸入時打錯——錯一個鹼基、貼錯行、或貼成另一個產品線的序列。



為什麼會踩

15 個鹼基的字串，人眼幾乎驗不出一字之差。

踩了會怎樣

reads 對不上 reference，那條 ADT 幾乎歸零。流程照樣跑完，一個錯都不報。

一個鹼基，整條 ADT 全掛

feature_ref.csv 裡 CD3 的 sequence 欄：

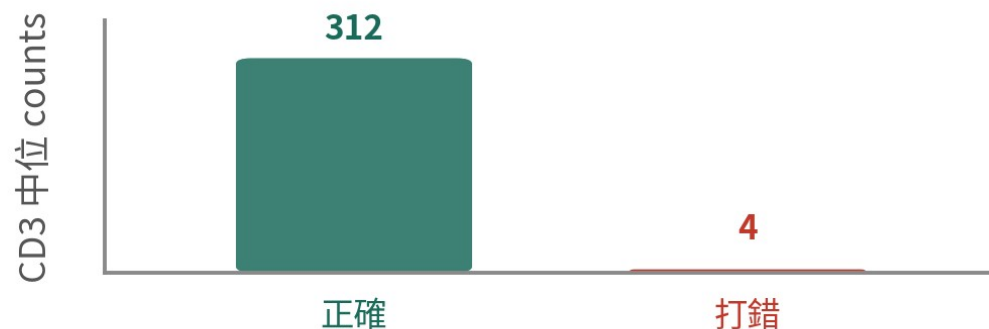
試劑說明書

A A C A A G A C C C T T G A G

你打進 CSV 的

A A C A A G A C C C T T G **C** G

差一個鹼基



reads 對不上 reference，整條 ADT 幾乎歸零
流程照樣跑完、不報任何錯

一行檢查：web_summary 的
「Antibody Reads Usable」異常低就要起疑

模擬資料

一行檢查：web_summary 的 Antibody Reads Usable 異常低就要起疑

雷三：hashing 的 negative 一律丟掉

雷是什麼

HTODemux 分完類，negative 全部 subset 掉，頭也不回。



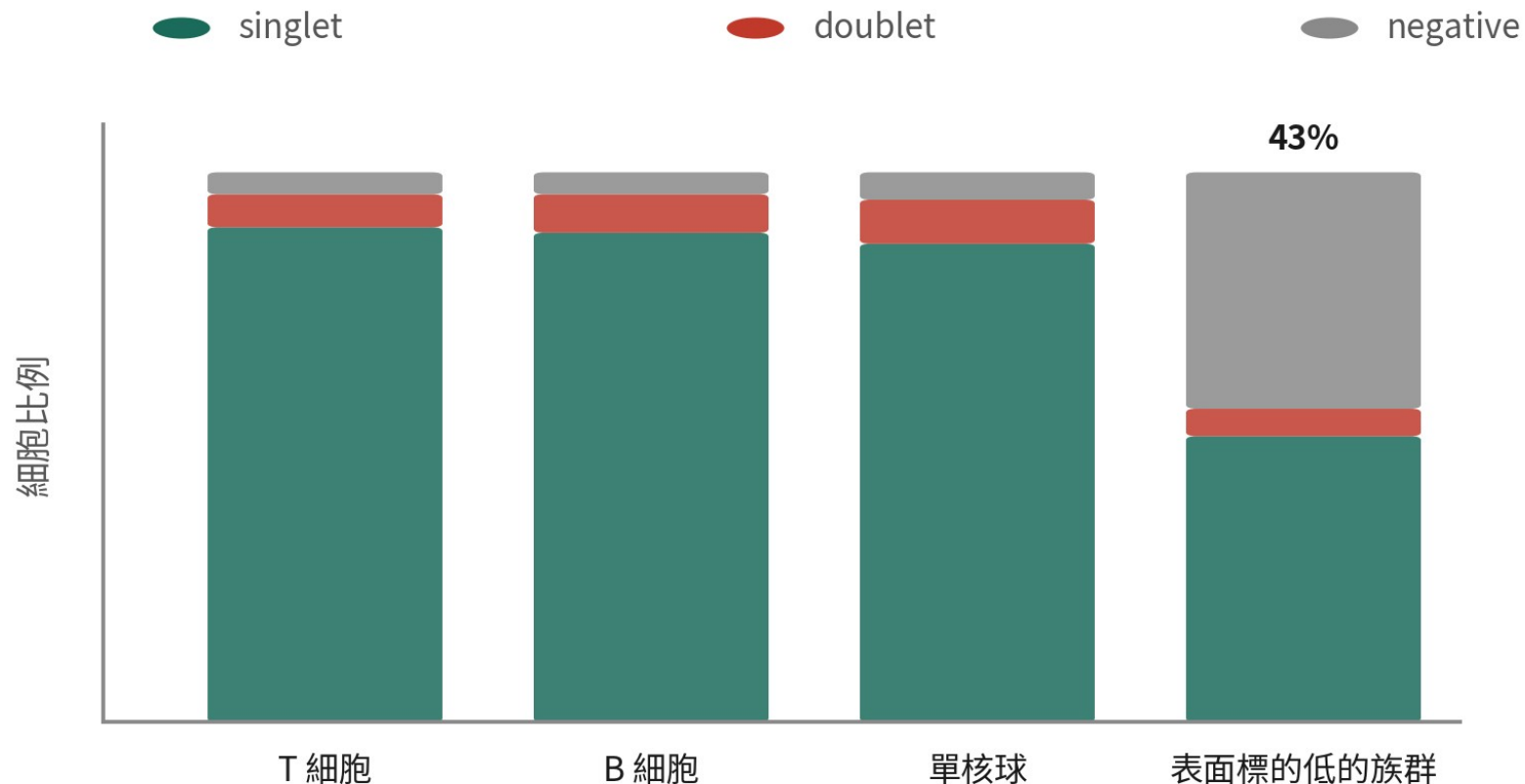
為什麼會踩

「沒標籤＝沒資訊」，聽起來天經地義。

踩了會怎樣

HTO 抗體認的表面蛋白在某些族群本來就低——它們整群被判 negative，一丟就是系統性偏差。

negative 不是均勻分布的



negative 全丟的代價

HTO 抗體認的是表面蛋白
(CD298/ β 2M) ——
這些蛋白低的族群染不上，
整群被判 negative

——一律丟掉 = 系統性丟掉
——整個細胞族群

模擬資料

一行檢查: `table(細胞型別, HTO 分類)` —— negative 集中在特定族群就要警覺

還有兩個小雷，順手記下

雷	後果與症狀	預防
GEX 與 FB 的 fastq 配對搞混	兩邊 mapping 率同時暴跌	libraries CSV 逐列對路徑
multi config 漏寫 [samples]	config 檢查直接報錯、跑不起來	先跑 multi 的 config 檢查

踩雷區小結

**上游的錯誤大多不報錯，
只留下異常安靜的數字。**

web_summary 的三個位置：aggregation 表、Antibody Reads Usable、per-sample 細胞數。每次都看。

⑤ +

複雜樣本策略室

上游的指令你都會了——現在想像你手上不是四管 PBMC，是一顆腫瘤

多區域取樣：core、margin 與配對



一顆腫瘤，三種生態——
只取一管，只看得到一種

GSE84465 (Darmanis 2017)：四位 GBM 病人 × 核心／邊緣



8 個樣本 · 3,589 顆細胞 · $n = 4$ 對
配對 = 病人間差異被扣掉，功效大增

設計期教訓

邊緣的惡性細胞
總共只有 31 顆——
「惡性 core vs margin」
這一題問不了

→ 動刀前先估每部位 ×
每型別能回收幾顆

配對紅利

同病人多區域 = 同一套 CNV
合併分析比跨病人單純 (B11)

示意；樣本與細胞數出自 GSE84465

GSE84465：四位 GBM 病人 × 核心／邊緣配對——邊緣只有 31 顆惡性細胞，這個教訓要在設計期學

hashing 到了腫瘤，前提開始鬆動

雷三的 negative 偏差，在腫瘤被放大好幾倍

解離壓力



實體瘤要長時間酵素消化，表面蛋白受損，negative 率整體上升

抗原表現差異



惡性細胞的 CD298 / β 2M 隨病人與 CNV 而異——偏差正好落在你最關心的細胞上

凍存與單核



細胞核沒有表面蛋白，抗體 hashing 出局；改核專用標記或 genotype demultiplexing

治療前後設計： **confound** 在設計期擋掉

整合修得掉「分佈不齊」，修不掉「完全重疊」

危險設計

治療前一批做完，治療後另一批

- 批次與治療效應完全重疊
- B11 的整合無從分辨兩者
- 統計救不了設計

設計期預防

配對貫穿、批次打散

- 同病人治療前後配對分析
- 每一批都同時含前與後的樣本
- 可行時混樣同 channel—— 批次歸零

本集 checklist



多 lane 交給 count；多樣本先各跑 count



合併預設走 R 端；aggr 只在需要單一矩陣時用



用 aggr 時，想清楚 --normalize 要不要關



CellPlex 或 VDJ → multi；ADT/HTO → count+FB 也可



feature reference 的序列逐字核對過



hashing 的 negative 看過分布再決定去留

六格全勾，上游段就畢業了

一句話總結

**count 管一個樣本， multi 管一個實驗；
合併，預設留給下游。**

能把這句話講給同學聽，這一集就到位了。

自測 1

cellranger aggr 預設的 --normalize=mapped 做了什麼？

- A 把每個細胞的 UMI 數縮放到同一總量
- B 把每個樣本的 reads 抽樣到最淺樣本的深度
- C 移除批次效應
- D 把 barcode 加上 -1、-2 的字尾

答 B。它在「reads」層級抽樣，把所有樣本向下拉平到最淺的深度，抽掉的 reads 不再出現在任何輸出。它不是批次校正；字尾則是 aggr 一定會做的事，與 normalize 無關。

自測 2

下列哪種實驗「一定」要走 **cellranger multi** ?

- A 四個樣本各自跑完 count 要合併
- B GEX 加 CITE-seq 抗體 (ADT)
- C CellPlex (CMO) 多重化的樣本
- D 同一個 library 分四條 lane 定序

答 C。CMO 的樣本歸屬只能由 multi 拆；A 用 aggr 或 R 端合併，B 用 count 加 libraries CSV 也行，D 是 count 自動處理。

自測 3

HTODemux 跑完，某類細胞的 negative 比例是其他族群的十倍。正確的反應是？

- A negative 本來就要丟，照丟
- B 懷疑那個族群染不上 HTO 抗體，先查再決定去留
- C 提高 positive.quantile 讓 negative 變少
- D 重跑一次 HTODemux

答 B。 HTO 抗體認的是特定表面蛋白，表現低的族群整群染不上。集中在單一族群的 negative 是生物性訊號，不是雜訊；直接丟或調參數蓋掉，都是把偏差藏起來。

下集預告

B5：讀入資料與 QC

上游到此為止——你手上已經有 filtered matrix 了。

但 Cell Ranger 說的「細胞」，Seurat 不一定承認。下一集開始進 R：哪些細胞該丟，誰說了算？